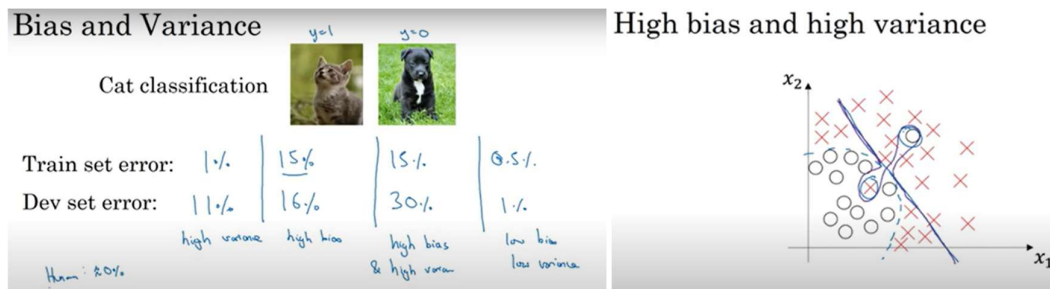
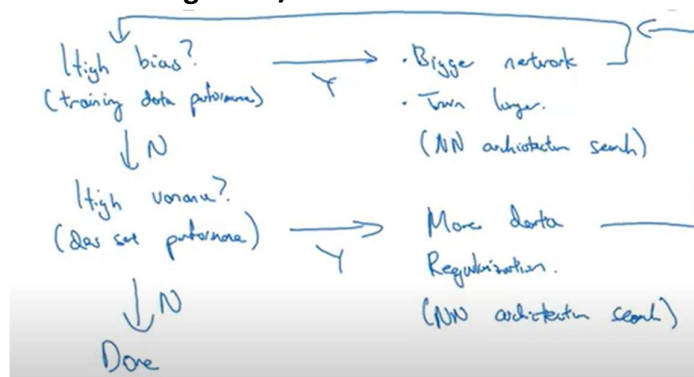


Course 2: Improving Deep NNs

- Data is split into: -
 - Training Dataset – Used to train the model
 - Development Dataset – Used to check which model works best on this dataset
 - Testing Dataset – Used to test the best model
- The ratio of splitting the dataset depends on many factors and not a constant value. If we have a very large dataset, we would need to allocate only a small percentage of the total data to testing and dev sets.
- Test & Dev datasets should come from the same set as Train dataset
- **High Bias** happens when the dataset is underfitted resulting in weak classification.
High Variance happens when the dataset is overfitted and leads to a worse accuracy in dev set than in the training set.



- **How to tackle high bias/variance?**



- **Regularization:**

For cost function:

$$J(w^{(0)}, b^{(0)}, \dots, w^{(L)}, b^{(L)}) = \frac{1}{m} \sum_{i=1}^m \ell(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{(l)}\|^2$$

$$\|w^{(l)}\|^2 = \sum_{i=1}^{n^{(l-1)}} \sum_{j=1}^{n^{(l)}} (w_{ij}^{(l)})^2 \quad w: \begin{pmatrix} n^{(l-1)} & n^{(l)} \end{pmatrix}$$

"Frobenius norm"

For back-propagation:

$$dW = (\text{from backprop}) + \frac{\lambda}{m} W^{(l)}$$

$$W^{(l)} := W^{(l)} - \lambda dW^{(l)}$$

This is also called "weight decay"

- Regularization **penalizes** the model to **avoid** it from **adding large weight values**. This **prevents** the model from **learning complex features** and hence, avoids overfitting.
- **Dropout Regularization** involves randomly dropping a few neurons from the network while training. This helps the model in learning few generalized features and avoids getting overfitted. Dropouts are only done while training and avoided during testing.
Downside: The cost function is now not well defined.

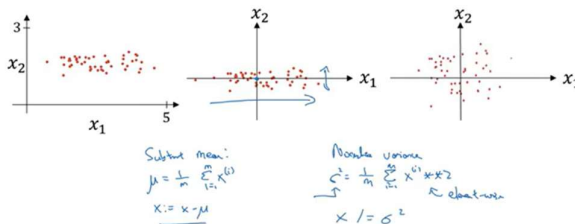
Illustrate with layer $l=3$. keep-prob = 0.8

$$d3 = \text{np.random.rand}(a3.\text{shape}[0], a3.\text{shape}[1]) < \text{keep-prob}$$

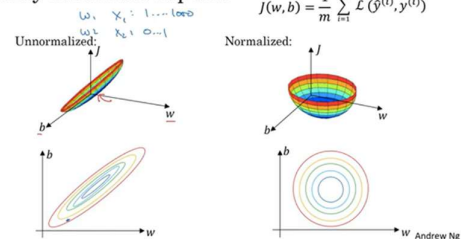
$$a3 = \text{np.multiply}(a3, d3) \quad \# a3 \neq d3.$$

$$a3 /= \text{keep-prob}$$

- **Data Augmentation** is another regularization method which involves adding cropped, skewed, blurred, or distorted images of the training set to train the model.
- **Early Stopping/Orthogonalization** is when we stop the training mid-way to achieve a better accuracy on the dev set.
- **Input is Normalized** by subtracting its mean and dividing by variance. This improves the gradient descent and helps in training the model in lesser iterations.



Why normalize inputs?

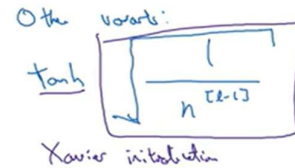


- In a very large NN, if the weights are slightly greater than 1, it results in exploding gradients and if the weights are slightly less than 1, it results in vanishing gradients.

- **Weight Initialization:-**

$$W^{[1]} = n_p \cdot \text{random} \cdot \text{random}(\text{shape}) * n_p \cdot \sqrt{\frac{2}{n^{[1-1]}}}$$

ReLU $g^{[1]}(z) = \text{ReLU}(z)$



- Use two-sided difference while calculating gradients as it reduces error.

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$$

- **Gradient Checking** is a method for debugging the gradient calculation by approximating the grads. Expected grad can be found by forming a single vector of all the parameters (W_i, b_i) called θ and taking 2-sided difference.

$$d\theta_{\text{exp}} = ((\theta + \epsilon) - (\theta - \epsilon)) / 2\epsilon$$

While applying grad-check, we still need to remember regularization if we have used it. Grad check cannot be used with dropout regularization as random nodes will be turned off. We'd need to remove dropout and then apply grad-check.

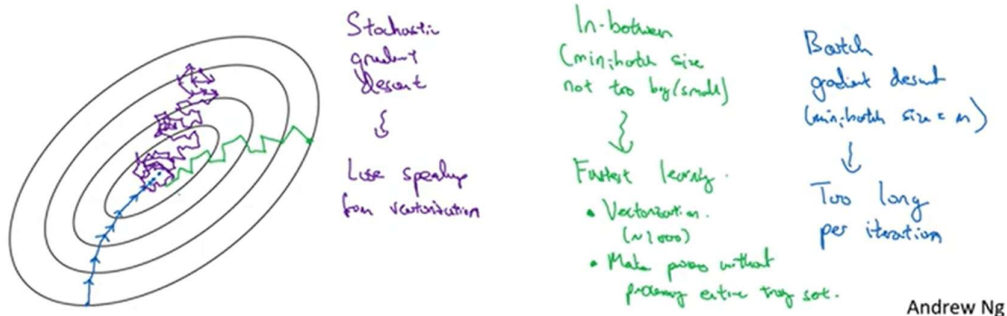
- **Mini-Batch Gradient Descent** involves splitting the large data-set into smaller batches and running gradient descent over the batches independently. Gradient descent in mini-batch is a bit noisier than batch grad descent.

Choosing your mini-batch size

→ If mini-batch size = m : Batch gradient descent. $(X^{[1]}, Y^{[1]}) = (X, Y)$

→ If mini-batch size = 1 : Stochastic gradient descent. Every example is its own $(X^{[1]}, Y^{[1]}) = (x^{(1)}, y^{(1)}) \dots (x^{(n)}, y^{(n)})$ mini-batch.

In practice: Somewhere in-between 1 and m



Preferably choose batch size in powers of 2 (64, 128, 512, 1024) only for larger NNs (with $m > 2000$).

- **Exponentially Weighted Average:** $v_t = \beta v_{t-1} + (1-\beta)\theta_t$
 β is a parameter that controls how wide of a dataset is considered while averaging.
 The data points now will be averaged across approximately $\frac{1}{1-\beta}$ days.

- When the β value is higher, the initial resultant values are way lower than what is expected. A simple way to overcome this is by using a **Bias Correction**.

$$\text{Bias Correction} = \frac{v_t}{1-\beta^t}$$

The denominator here, at lower values, makes the equation much similar to averages of the first t values. At higher values of t , β^t becomes negligible and equation returns to previous form.

- **Gradient Descent with Momentum:**

On iteration t :

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1-\beta)dW$$

$$v_{db} = \beta v_{db} + (1-\beta)db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- **RMS Prop Algo:**

On iteration t :

Compute dW, db on current mini-batch

$$S_{dW} = \beta S_{dW} + (1-\beta) \frac{dW^2}{\text{element-wise}}$$

$$S_{db} = \beta S_{db} + (1-\beta) db^2$$

$$W := W - \alpha \frac{dW}{\sqrt{S_{dW}}} \quad b := b - \alpha \frac{db}{\sqrt{S_{db}}}$$

The denominator here dampens the oscillations in direction with larger oscillations and doesn't affect the other directions much.

- **Adam Optimization:** (Merges both momentum and RMS Prop)

$$V_{dW} = 0, S_{dW} = 0, V_{db} = 0, S_{db} = 0$$

On iteration t :

Compute dW, db using current mini-batch

$$V_{dW} = \beta_1 V_{dW} + (1-\beta_1)dW, \quad V_{db} = \beta_1 V_{db} + (1-\beta_1)db \quad \leftarrow \text{"momentum"} \beta_1$$

$$S_{dW} = \beta_2 S_{dW} + (1-\beta_2)dW^2, \quad S_{db} = \beta_2 S_{db} + (1-\beta_2)db^2 \quad \leftarrow \text{"RMSprop"} \beta_2$$

$$V_{dW}^{\text{corrected}} = V_{dW} / (1-\beta_1^t), \quad V_{db}^{\text{corrected}} = V_{db} / (1-\beta_1^t)$$

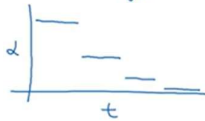
$$S_{dW}^{\text{corrected}} = S_{dW} / (1-\beta_2^t), \quad S_{db}^{\text{corrected}} = S_{db} / (1-\beta_2^t)$$

$$W := W - \alpha \frac{V_{dW}^{\text{corrected}}}{\sqrt{S_{dW}^{\text{corrected}} + \epsilon}} \quad b := b - \alpha \frac{V_{db}^{\text{corrected}}}{\sqrt{S_{db}^{\text{corrected}} + \epsilon}}$$

**Adam stands for
Adaptive Moment
Estimation**

- **Learning Rate Decay** implies reducing the learning rate as the training progresses. Initial high α converges the model faster and the lower α at the end ensures that the model will be closer to the global minima.

Ways to apply this:

- $\alpha = \frac{1}{1 + \text{decay-rate} * \text{epoch-num}} \cdot \alpha_0$
- Exponential Decay: $\alpha = 0.95^{\text{epoch-num}} \cdot \alpha_0$
- $\alpha = \frac{k}{\text{epoch-num}} \cdot \alpha_0$ or $\frac{k}{\sqrt{t}} \cdot \alpha_0$
- Discrete Staircase: 
- Manual Decay

- α is one of the most important hyperparameter to be tuned. Followed by mini-batch-size, β , and number of hidden layers.

For hyperparameter tuning, choose random values and try, don't follow grid.

- Choosing right scale is important while tuning hyperparameters. Logarithmic scale can be taken by picking a random number r (using `random.rand()`) and then taking 10^r .

- **Batch Normalization** involves normalization (subtracting mean and dividing by variance) of Z vector after every layer before calculating a .

$$z_{\text{norm}} = \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z} = \beta z_{\text{norm}} + \gamma$$

β and γ are tunable parameters like weights and biases. In batch norm, b (bias vector) can be ignored.

Batch Norm makes it easier to train the later layers much similar to how Normalization of input affects training. Also, Batch norm has slight regularization effects.

- **Softmax Classifier** is used to classify into more than one classes.

$$a_i = \frac{e^z_i}{\sum e^z_j}$$

- The derivative can be zero at local minima or saddle points too, apart from global minima. Plateaus are regions where the derivative is almost zero, so this makes the training slower.